COMP 2510 – Week 7 In-Class Activity

Name: Aika Manalo

Instructions

- Complete the activities during the lecture. Please copy paste this the template below for new activities.
- Paste your code screenshots and type your answers in the sections below.
- Save as **Lastname_Firstname_W7_Activity.pdf** and upload.

Activity X:

1) Last week we learnt that static limits visibility for functions to the current file. Does it have other powers? What do you expect to be the output of this code?

```
*/
void counter(void)
{
    static int count = 1;
    printf("This has been called %d time(s)\n", count);
    count++;
}

int main(void)
{
    counter();
    counter();
    counter();
}
```

```
Run   inClassActivity
/Users/aika/Documents/areas/cst/202610_t2/comp2510/lecture/lectures/cmake-build
This has been called 1 time(s)
This has been called 2 time(s)
This has been called 3 time(s)

Process finished with exit code 0
```

- in a normal function, when a function ends, it gets cleaned up and thrown away
- count is declared static -> gains a permanent memory when the function ends, count stays exactly where it is and waiting for the next time you call the function
- most local variables live on the stack, but static variables live in the data segment
- if count were a normal variable, it would be set to 1 every single time counter was called. Since count is a static variable, it only gets initialized ONCE (the first time counter gets called) and initialization after every subsequent call gets ignored -> it holds its state
- even if the variable count lives for the entire program, its scope is still contained inside the counter() function -> encapsulation
- if you declare a function OR a variable static in the global scope (top of your file. without any surrounding brackets) it belongs to THIS file only. no other files (.c) in this project can see it, or use it -> like private in Java

2) What is the size of this struct? Why?

```
struct Mystery {
    char a;   // 1 byte
    int b;    // 4 bytes
    char c;   // 1 byte
};
```

12 bytes (on modern 32/64-bit systems); Why? -> CPU doesn't read memory 1 byte at a time but instead it grabs chunks (like 4 or 8 bytes)

If int is split across two different chunks, CPU needs to do two reads and some math <- makes program slow. To stay fast, compiler adds padding (empty space) so int starts exactly where the CPU expects it.

Note: different computers might add padding differently. Not accounting for those "empty spaces" (padding), the data gets shifted, and program starts to read the wrong numbers -> ruining File I/O (saving/loading) and Networking (sending data).

Therefore, if char a is 1 byte.... + 3 bytes to align. Total = 4
int b is 4 bytes... already aligned. Total = 8
char c is 1 byte.... + 3 bytes to align. Total = 12

3) You have a system that currently uses float for currency, but you suspect you might need to change it to double later. Is there a way to address it in one place? How?

```
typedef float currency_t; // Q3.A

struct Student // Q3.B
{
    char name[50];
    int id;
};

// Have to write 'struct' every time...
struct Student s1;
struct Student s2;

// declare it as a typedef instead...
typedef struct Student
{
    char name[50];
    int id;
} Student_t;

// Cleaner
Student_t s3;
Student_t s4;
```

Yes. Using 'typedef' <- think of it as an "alias" or "nickname" -> lets you create a new name for an existing data type.

<u>*Q3.A*</u>: This specific example shows how 'typedef' is used to create Abstraction.
instead of writing 'float price;' all over the code, you define your intent once at the very top.
<u>*Q3.B*</u>: Can make DRYer code with structs -> basically a shortcut

Note: naming convention suffix: '_t' <- stands for "Type" (size_t or uint32_t <- C's own built-in types also use this)

4) **We use printf to write to the screen. Conceptually, C treats the screen as a "Stream". What is the name of this standard output stream? What are the other streams that are open by default?**

stdout: used for writing normal output to the ***terminal*** or piped to another command

- The normal outbound pipe; where program sends its regular messages (usually ***terminal***)

stdin: used for reading input, usually from ***keyboard*** or piped from another command

- The inbound pipe; where program listens for data (usually ***keyboard***)

stderr: used for writing error messages or diagnostics

- The emergency outbound; dedicated lane for **error messages or warnings**

Two Outbound Pipes: stdout vs. stderr → piping output to a file uses stdout; if your program crashes, you don't want the error message hidden inside that file, hence, stderr is a separate pipe.

printf("hi"); is essentially a shortcut for **fprintf(stdout, "hi");** → programmers got lazy typing stdout

5) **When we ask the OS to open a file using fopen("data.txt", "r"), it returns a FILE*. However, if the file doesn't exist, what does it return, and why is checking for this value critical?**

```
FILE *fp = fopen( filename: "test.txt",  mode: "r");
if (fp == NULL)
{
    fprintf(stderr, "Error: File not found!\n"); // use stderr for errors
    return 1;
}
```

Returns NULL; dereferencing a NULL pointer → undefined behaviour. Checking allows for graceful error handling.

FILE* is a pointer that tells you exactly where that file you're referencing is. If file doesn't exist or is restricted -> fopen fails and returns NULL.
Need to check, why? → if you don't check if the file opened successfully and immediately try to read from it, you are DEREFERENCING A NULL POINT and likely crash (segmentation fault).

6) **What are some ways to read an unformatted text file?**
REMEMBER TO USE <u>fclose()</u> AT THE END OF EACH FUNCTION ← release resources

a) use **fgets** → returns a pointer to the character array where the read data is stored
- it grabs a whole line of text at once and dumps it into a buffer (temp storage area)
- you give it a chunk of mem and tell it to keep scooping until it hits a new line or runs out of empty space
- why use? Fast → instead of going back and forth to the file for every letter, you take big gulps; most common way to read text files line-by-line
- however…n need to make sure your buffer is big enough or it'll get cut off

```c
char buffer[1000];
while (fgets(buffer, sizeof(buffer), fp) != NULL)
{
    printf("%s\n", buffer);
}
fclose(fp);

int c; // type is int to accomodate EOF
while ((c = fgetc(fp)) != EOF)
{
    printf("%c", c);
}
```

b) use **fgetc** → read one character at a time until we reach end of file
- returns int to accommodate the EOF marker (eof signal is a number)
- great if need to inspect. Every single character (e.g. how many time letter 'e' appears)

```c
int c; // type is int to accomodate EOF
while ((c = fgetc(fp)) != EOF)
{
    printf("%c", c);
}

fclose(fp);
```

If file **<u>IS FORMATTED</u>**: use **fscanf** → skips leading whitespace and works similarly to **scanf**
- returns the number of input items successfully matched and assigned to variables
- safest to read until number matches the expectation + check reason not no match after
- looks for specific patterns (like nums or words) and skips off spaces/tabs
- use if unformatted file has some structure (like list of ages/id)
- however, if it expects a num, but finds a word "hello" → gets stuck
- hence why we include feof and ferror checks at the end to see if it stopped because the file ended or because it's clogged.

```c
// Loop continues as long as fscanf successfully reads one integer
while (fscanf(file, "%d", &number) == 1) {
    // Process the read number
    printf("Read number: %d\n", number);
}

// Check if the loop terminated due to EOF or a matching failure
if (feof(file)) {
    printf("Reached end of file.\n");
} else if (ferror(file)) {
    printf("An input error occurred.\n");
} else {
    printf("Matching failure (e.g., non-numeric input encountered).\n");
}
```

7) **If you want to create a new file or overwrite an existing one, we would pass "w" to fopen. What happens to previous content if the file exists?**
Opening an existing file in "w" mode will instantly truncate that file to 0 bytes. The computer the old content in the file first and you're given a "fresh file"

"w" = Write (Wipeout Rule)

- **File exists?** Pc truncates it to 0 bytes
- **Doesn't exist?** Pc just creates a new empty file for you

Example use case of: **fputc, fprintf, fputs**…

- If you want to keep existing content and add to the end, use **"a"** (append) mode instead
- **fputc**: writes a single character
- **fputs**: writes a whole string (e.g. a sentence)
- **fprintf**: crazy powerful, works like printf and lets you plug in variables like integers using format specifiers

```
FILE *fp;
int x = 32;

fp = fopen( filename: "output.txt",  mode: "w");

fputc('B', fp);
fputc('\n', fp);
fprintf(fp, "x = %d\n", x);
fputs("Hello, world!\n", fp);

fclose(fp);
```